

Generative AI · [Follow publication](#)

★ Member-only story

Our AI Has a Perfect Memory and Never Hallucinates. Here's How We Built It.

5 min read · Nov 7, 2025



Arturo García

Follow



Listen



Share



More

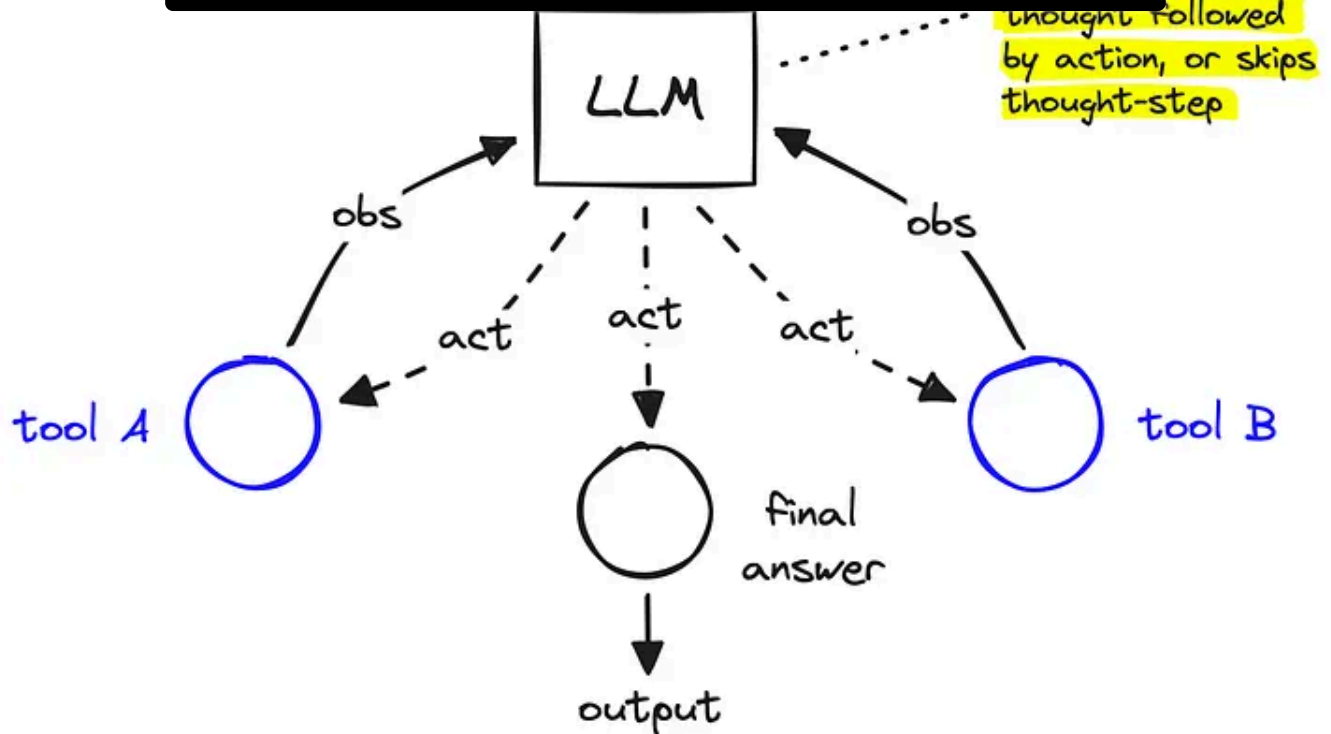
This is Part 3 of our “Building in Public” series. We’ve talked about our architecture and our stack. Now, we’re opening up the brain of our AI to show you how we solved the two biggest problems in personal AI: trust and context.

The promise of a personal AI is intoxicating. But it hinges on one non-negotiable principle: trust. An AI that hallucinates — confidently inventing facts — is a novelty. An AI that hallucinates about *your life* is a nightmare. It corrupts your memory, breaks your trust, and renders the entire concept useless.

From day one, our mission was to build an AI that could be a true thought partner. And a true partner needs two things: a perfect memory of what you’ve told it, and the intelligence to understand context. Simply plugging a Large Language Model (LLM) into a database doesn’t work. It requires a sophisticated, stateful architecture.

The core of this architecture is **LangGraph**, a framework for building AI agents. Think of it as the central nervous system of our AI. Instead of a simple, linear chain of commands, LangGraph allows our AI to think, make decisions, and use a specialized toolkit to answer complex questions.

Welcome back. You are signed into your member account
bg....@jaxondigital.com.



Langgraph flow example

This is the story of that toolkit.

The Problem: Humans Don't Talk Like Computers

A user won't ask, "Retrieve entries between 2024-08-01 and 2024-08-31 with a sentiment score below 0.2." They'll ask:

> "I felt pretty down last August. What was going on?"

This simple question contains multiple layers of complexity:

1. **Temporal Ambiguity:** What does "last August" mean?
2. **Semantic Ambiguity:** What does "feeling down" look like in the text?
3. **Causality:** What events or themes ("what was going on") are linked to that feeling?

To solve this, our AI can't just be a passive text generator. It has to be an agent with a set of specialized **Tools**.

Tool #1: The Time-Traveling Interpreter (Temporal Query Tool)

The first tool our agent often reaches for is the `TemporalQueryTool`. Its only job is to translate natural language queries into a structured format that the LLM can understand.

Welcome back. You are signed into your member account
bg...@jaxondigital.com.

We use an LLM for this, but in a very controlled way. We give it a specific task and a strict output format.

```
from langchain_core.tools import tool
from datetime import datetime
@tool
def temporal_query_parser(natural_language_query: str) -> dict:
    """
    Parses a natural language query to extract a precise start and end date.
    The LLM is instructed to use today's date ({current_date}) as a reference
    and return a JSON object with 'start_date' and 'end_date'.
    """
    # In our real app, this function would contain a specific,
    # fine-tuned prompt sent to an LLM.
    # For example, "last August" would be converted to:
    # {"start_date": "2024-08-01", "end_date": "2024-08-31"}
    # This is a mock implementation for clarity.
    if "last august" in natural_language_query.lower():
        return {"start_date": "2024-08-01", "end_date": "2024-08-31"}
    # ... more logic ...
    return {}
```

When our agent sees “last August,” it doesn’t guess. It calls this tool, gets a structured `{“start\_date”: “2024-08-01”, “end\_date”: “2024-08-31”}` response, and adds this precise information to its “scratchpad” for the next step.

Tool #2: The Meaning-Based Search Engine (RAG Vector Search)

This is the core of the AI’s “perfect memory,” which we touched on in the last article, but now it’s a tool in our agent’s belt. When the agent needs to find relevant information, it uses the `VectorSearchTool`.

This tool takes the user’s query (e.g., “feeling down”) and the structured data from other tools (like the date range from our `TemporalQueryTool`) and performs a filtered vector search on the user’s journal entries.

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

```
from pgvector import Pgvector
@tool
def vector_search_journal(query: str, start_date: str = None, end_date: str = None) -> list[str]:
    """
    Performs a semantic vector search on the user's journal chunks.
    Can be filtered by a date range.
    """
    query_embedding = embedding_model.encode(query)
    # We build a query that filters by date first, then performs the vector search.
    # This is highly efficient in PostgreSQL with pgvector.
    chunks = EntryChunk.objects.filter(
        entry__created_at__range=(start_date, end_date)
    ).order_by(
        L2Distance('embedding', query_embedding)
    )[:5] # Get the top 5 most relevant chunks
    return [chunk.chunk_text for chunk in chunks]
```

The agent now has the raw material: the most relevant text snippets from the correct time period. But it's not ready to answer yet.

The Conductor: How LangGraph Orchestrates the Workflow

This is where it all comes together. LangGraph defines the decision-making process. For our query, “I felt pretty down last August. What was going on?”, the flow looks like this:

1. **Start Node:** The query enters the graph.
2. **Router Node:** An LLM-powered router examines the query and the available tools. It identifies two key tasks: parsing the date (“last August”) and searching for content (“feeling down,” “what was going on”). It decides the first logical step is to parse the date.
3. **Tool Call:** The graph executes the `temporal\_query\_parser` tool. The tool returns the date range.
4. **State Update:** The graph's state is updated. It now contains the original query AND the structured date range.
5. **Router Node (Again):** The state is passed back to the router. The router sees it has a date range and still needs to find content. It now decides to call the

`vector\_search\_journal` tool, passing both the query and the date range as arguments.

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

6. **Tool Call:** The graph executes the search tool. A list of relevant text chunks is returned.
7. **State Update:** The state is updated again. It now contains the query, the date range, and the retrieved context.
8. **Router Node (Final Pass):** The router sees it has all the necessary information to answer the question. It decides to route to the final `Generate` node.
9. **Generate Node:** This is our final, non-hallucination guardrail. It takes all the retrieved context and uses a strict prompt, similar to our last article, to synthesize a final answer. The prompt essentially says: **“Based ONLY on the following text, summarize what was happening.”**

This stateful, multi-step process allows our AI to break down a complex human question into a series of logical, machine-executable steps. It ensures that every single assertion in the final answer is directly traceable to the user’s own words.

We’re not just building a product; we’re pioneering a new level of trustworthy, personal AI. It’s a complex system, but it’s the only way to deliver on our promise: an AI that truly knows you, and that you can truly trust.

In the next post, we’ll zoom out and share our complete “Bootstrapper’s Tech Stack” — all the tools, services, and technologies we use to stay lean and competitive.

This story is published on [Generative AI](#). Connect with us on [LinkedIn](#) and follow [Zeniteq](#) to stay in the loop with the latest AI stories.

Subscribe to our [newsletter](#) and [YouTube](#) channel to stay updated with the latest news and updates on generative AI. Let’s shape the future of AI together!

Welcome back. You are signed into your member account
bg....@jaxondigital.com.



Follow

Published in Generative AI

71K followers · Last published 13 hours ago

Stay updated with the latest news, research, and developments in the world of generative AI. We cover everything from AI model updates, comprehensive tutorials, and real-world applications to the broader impact of AI on society. Work with us: jimclydegm@gmail.com



Follow

Written by Arturo García

293 followers · 126 following

AI Engineer/Analyst Learning while developing EchoesJournal. What to contact?
echoesjournal02@gmail.com :)

Responses (15)



Bgerby

What are your thoughts?



Hendrik Leier

3 days ago



The title is a bit misleading. Yes this is a grounded AI even a very grounded AI. But it is fundamentally still powered by LLMs which can and will make mistakes given enough sampling. It's another question how often

they actually do. I would have liked to see some numbers here



Welcome back. You are signed into your member account
bg....@jaxondigital.com.



Michael lantosca

5 days ago



OK, for a very limited and small application you're getting a bit closer to deterministic model. Now how is that scaled and maintained in a practical way for a large domain to build a deterministic model without abstracting facts as a manageable... [more](#)



1 reply

[Reply](#)



Andreas Exner

3 days ago



it's great to see how many effort you put into your architecture to minimize hallucinations and maximize trust. But Hendrik already mentioned that your headline is misleading. LLM will always hallucinate no matter what you're building to mitigate the... [more](#)



1 reply

[Reply](#)

[See all responses](#)

More from Arturo García and Generative AI

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

 Arturo García

How to build an AI Startup project in real life?

This is Part 1 of our “Building in Public” series, where we’re documenting the messy, unfiltered journey of creating an AI tool for solo...

✦ Oct 30 🖱 2



 In Generative AI by Gao Dalie (高逵烈)

DeepSeek-OCR + LLama4 + RAG Just Revolutionized Agent OCR Forever

During the weekend, I scrolled through Twitter to see what was happening in the AI community. Once again, DeepSeek has drawn worldwide...

★ Oct 27



Welcome back. You are signed into your member account
bg....@jaxondigital.com.



In Generative AI by Joe Njenga

15 Ways I'm Using Google's Nano Banana for UI/UX Design (Like a Pro)

I am not a UI/UX designer, but Google Nano Banana is filling the gap, and that doesn't mean I'm becoming one overnight.



Oct 9



533



7



Arturo García

Why Serverless Was the Wrong Choice for Our AI Business Project

This is Part 2 of a series of articles about the challenges of building a serverless AI business project. The main lesson: not all architectures are created equal.

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

★ Nov 4 🖱️ 2



See all from Arturo García

See all from Generative AI

Recommended from Medium

👤 Dr Adrian Lucas Malec

How I Built Lightning-Fast Vector Search for Legal Documents

From 53 to 2,880 queries/sec with USearch

Oct 19 🖱️ 395 💬 5



Welcome back. You are signed into your member account
bg....@jaxondigital.com.

 In AIGuys by Vishal Rajput 

Kimi K2 Just Killed OpenAI, & Claude

If one thing tech bros are better than at building AI, it is creating hype around AI. And the release of Kimi K2 just proved the opposite...

✦ Nov 8 🖱 515 💬 17



 In Medialesson by Marius Schröder

JSON vs TOON—A new era of structured input?

Why structure matters more than ever

Nov 3 🖱️ 79%



Welcome back. You are signed into your member account
bg....@jaxondigital.com.

 Arturo García

How to build an AI Startup project in real life?

This is Part 1 of our “Building in Public” series, where we’re documenting the messy, unfiltered journey of creating an AI tool for solo...

★ Oct 30 🖱️ 2



 In AI Software Engineer by Joe Njenga

Anthropic Just Solved AI Agent Bloat—150K Tokens Down to 2K (Code Execution)

Anthropic just announced a major efficiency breakthrough by 98%, shift from tool calling to MCP code execution

Welcome back. You are signed into your member account [bg.....@jaxondigital.com](#).

★ Nov 6 🖱️ 655 💬 48  ⋮

 Ignacio de Gregorio

A New DeepSeek Moment?

Alibaba's Aegaeon has strong implications

★ 5d ago 🖱️ 903 💬 22  ⋮

See more recommendations